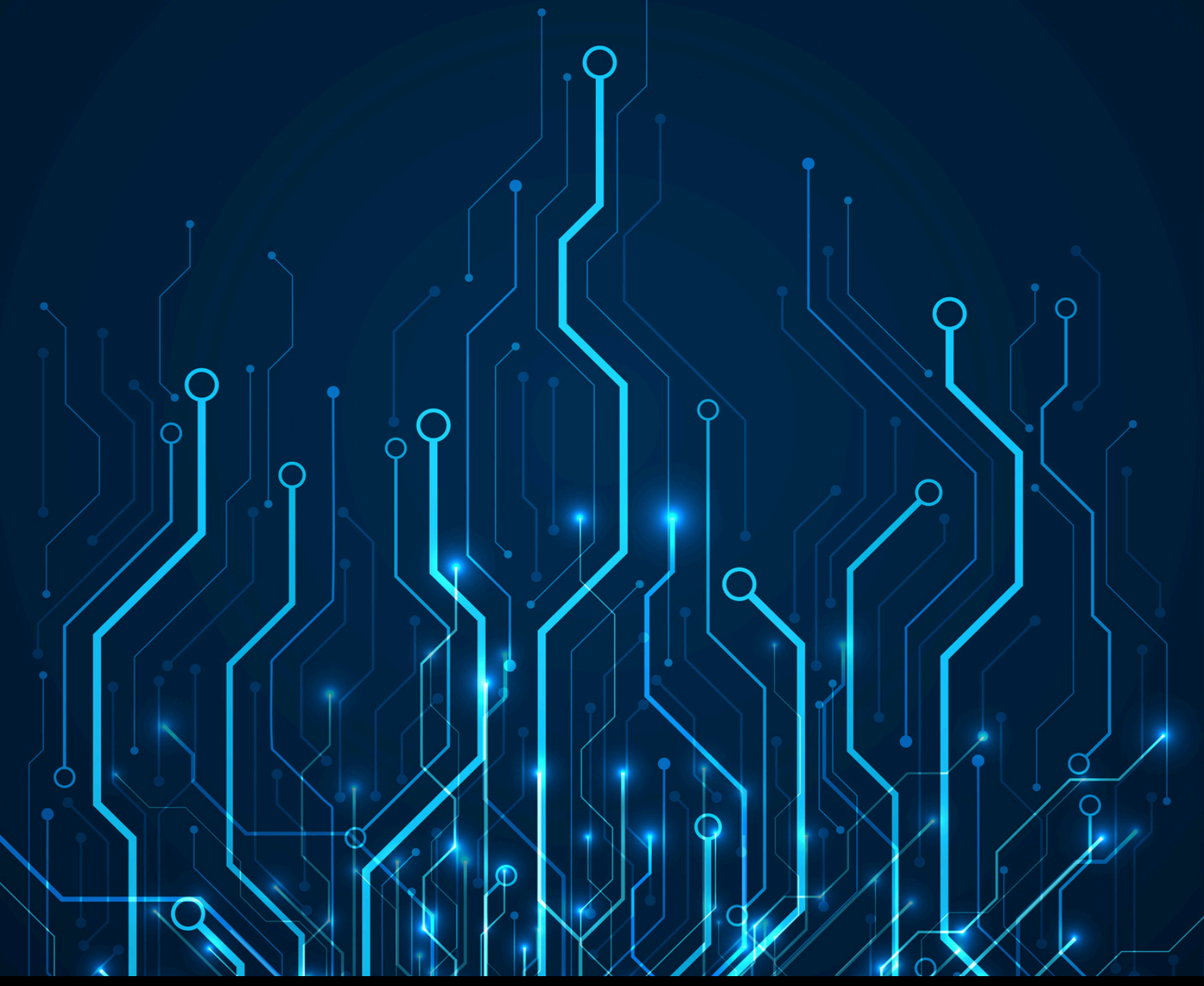
PROV LOGIC

TOP 50 VERILOG

INTERVIEW QUESTIONS ASKED IN VLSI COMPANIES



USEFUL INTERVIEW GUIDE FOR VLSI ASPIRANTS

1. Write Verilog RTL for a parameterized synchronous FIFO with full, empty, wr_en, and rd_en signals.

```
module sync_fifo #(parameter DEPTH=8, WIDTH=8)(
  input clk, rst,
  input wr_en, rd_en,
  input  [WIDTH-1:0] din,
  output reg [WIDTH-1:0] dout,
  output full, empty
);

reg [WIDTH-1:0] mem [0:DEPTH-1];
reg [$clog2(DEPTH):0] wptr, rptr;

assign full  = (wptr - rptr == DEPTH);
assign empty = (wptr == rptr);

always @(posedge clk or posedge rst) begin
  if(rst) begin
    wptr <= 0;
    rptr <= 0;
  end
  else begin
    if(wr_en && !full) begin
      mem[wptr[$clog2(DEPTH)-1:0]] <= din;
      wptr <= wptr + 1;
    end
    if(rd_en && !empty) begin
      dout <= mem[rptr[$clog2(DEPTH)-1:0]];
      rptr <= rptr + 1;
    end
  end
end
endmodule
```

2. Write a Gray code counter that outputs Gray instead of binary.

```
module gray_counter #(parameter N=4)(
  input clk, rst,
  output reg [N-1:0] gray
);
reg [N-1:0] bin;

always @(posedge clk or posedge rst) begin
  if(rst) begin
    bin  <= 0;
    gray <= 0;
  end
  else begin
    bin  <= bin + 1;
    gray <= bin ^ (bin >> 1);
  end
end
endmodule
```

3. Design a module that outputs rise=1 on rising edge and fall=1 on falling edge of sig

```
module edge_detect(input clk, sig, output reg rise, fall);
reg sig_d;

always @(posedge clk) begin
  sig_d <= sig;
  rise  <= (sig & ~sig_d);
  fall  <= (~sig & sig_d);
end
endmodule
```

4. Design a 4-request round robin bus arbiter.

```
module rr_arbiter(
  input clk, rst,
  input [3:0] req,
  output reg [3:0] grant
);

reg [1:0] last;

always @(posedge clk or posedge rst) begin
  if(rst) begin
    last <= 0;
    grant <= 0;
  end else begin
    grant <= 0;
    casex({req,last})
      default: begin
        if(req[(last+1)%4]) begin grant[(last+1)%4] <= 1; last <= (last+1)%4;
      end
        else if(req[(last+2)%4]) begin grant[(last+2)%4] <= 1; last <= (last+2)%4;
      end
        else if(req[(last+3)%4]) begin grant[(last+3)%4] <= 1; last <= (last+3)%4;
      end
        else if(req[last]) begin
           grant[last] <= 1;
           last <= last;
         end
      end
    endcase
  end
end
endmodule
```

5. Convert one-hot input to binary code.

```
module onehot_to_bin(input [7:0] onehot, output reg [2:0] bin);
always @(*) begin
  case(onehot)
    8'b00000001: bin = 3'd0;
    8'b00000010: bin = 3'd1;
    8'b00000100: bin = 3'd2;
    8'b00001000: bin = 3'd3;
    8'b00010000: bin = 3'd4;
    8'b00100000: bin = 3'd5;
    8'b01000000: bin = 3'd6;
    8'b10000000: bin = 3'd7;
    default:     bin = 3'd0;
  endcase
end
endmodule
```

6. Parameterized N-bit adder using generate

```
module nbit_adder #(parameter N=8)(
  input  [N-1:0] a,b,
  output [N-1:0] sum,
  output cout
);
assign {cout,sum} = a + b;
endmodule
```

7. Design a Binary to one-hot decoder

```
module bin_to_onehot(input [2:0] bin, output reg [7:0] onehot);
always @(*) begin
  onehot = 8'b00000001 << bin;
end
endmodule
```

8. Write a verilog code for synchronous resettable shift register

```
module shift_reg #(parameter N=8)(
 input clk,rst,si,
 output reg [N-1:0] q
);
always @(posedge clk) begin
 if(rst) q <= 0;
 else    q <= {q[N-2:0],si};
end
endmodule
```

9. Write a verilog code for Mealy FSM sequence detector for "1011"

```
module seq1011(input clk,rst,x, output reg y);
reg [2:0] state;
parameter S0=0,S1=1,S2=2,S3=3;

always @(posedge clk or posedge rst) begin
 if(rst) begin state<=S0; y<=0; end
 else begin
  case(state)
    S0: begin if(x) state<=S1; else state<=S0; y<=0; end
    S1: begin if(~x) state<=S2; else state<=S1; y<=0; end
    S2: begin if(x) state<=S3; else state<=S0; y<=0; end
    S3: begin if(x) begin state<=S1; y<=1; end else begin state<=S2; y<=0; end end
  endcase
 end
end
endmodule
```

10. Design a Clock divider by N

```
module clk_div #(parameter N=4)(
 input clk,rst,
 output reg clk_out
);
reg [$clog2(N)-1:0] count;

always @(posedge clk or posedge rst) begin
 if(rst) begin count<=0; clk_out<=0; end
 else begin
  if(count==N-1) begin count<=0; clk_out<=~clk_out; end
  else count<=count+1;
 end
end
endmodule
```

11. Design a simple glitch-free 2-input clock mux circuit

```
module clk_mux(input clk1, clk2, sel, output clk_out);
assign clk_out = (sel)? clk2: clk1;
endmodule
```

12. Write a verilog code for CSR register with bit-wise write mask

```
module csr_reg #(parameter WIDTH=32)(
 input clk,
 input [WIDTH-1:0] wdata,
 input [WIDTH-1:0] mask,
 input we,
 output reg [WIDTH-1:0] rdata
);
always @(posedge clk) begin
 if(we) rdata <= (rdata & ~mask) | (wdata & mask);
end
endmodule
```

13. Design a Priority encoder with valid flag using verilog

```
module pencoder_valid(input [7:0] d, output reg [2:0] y, output reg valid);
always @(*) begin
 valid = |d;
 casex(d)
  8'b1xxxxxxx: y=7;
  8'b01xxxxxx: y=6;
  8'b001xxxxx: y=5;
  8'b0001xxxx: y=4;
  8'b00001xxx: y=3;
  8'b000001xx: y=2;
  8'b0000001x: y=1;
  8'b00000001: y=0;
  default: y=0;
 endcase
end
endmodule
```

14. Write Verilog RTL for a parameterized barrel shifter performing logical left and right shift based on control

```
module barrel_shifter #(parameter N=32)(
  input  [N-1:0] data,
  input  [$clog2(N)-1:0] shift,
  input dir,                  // 0 = left, 1 = right
  output reg [N-1:0] out
);

always @(*) begin
  if(dir == 0) out = data << shift;
  else         out = data >> shift;
end
endmodule
```

15. Implement 32-bit ALU with operations: ADD, SUB, AND, OR, XOR, SLT, NOR, PASS-A

```
module alu(
 input  [31:0] a,b,
 input  [2:0]  op,
 output reg [31:0] y
);

always @(*) begin
  case(op)
    3'b000: y = a + b;
    3'b001: y = a - b;
    3'b010: y = a & b;
    3'b011: y = a | b;
    3'b100: y = a ^ b;
    3'b101: y = (a<b)?32'd1:32'd0;
    3'b110: y = ~(a|b);
    3'b111: y = a;
  endcase
end
endmodule
```

16. Design a verilog code for Signed vs unsigned comparator circuit

```
module signed_compare(input signed [7:0] a,b, output gt);
assign gt = (a>b);
endmodule
```

17. Design a pseudo-random number generator using 8-bit LFSR.

```
module lfsr8(input clk,rst, output reg [7:0] q);
wire feedback;

assign feedback = q[7] ^ q[5] ^ q[4] ^ q[3];

always @(posedge clk or posedge rst) begin
 if(rst) q <= 8'h1;
 else    q <= {q[6:0], feedback};
end
endmodule
```

18. Design a 2-stage pipelined 8×8 unsigned multiplier.

```
module pipe_mult(input clk,
  input  [7:0] a,b,
  output reg [15:0] y
);

reg [7:0] a_reg, b_reg;
reg [15:0] mult_reg;

always @(posedge clk) begin
  a_reg   <= a;
  b_reg   <= b;
  mult_reg <= a_reg * b_reg;
  y <= mult_reg;
end

endmodule
```

19. Write a verilog code to drive a bus when en=1, else set high-impedance.

```
module tristate(
 input en,
 input [7:0] data,
 output [7:0] bus
);
assign bus = (en)? data : 8'bz;
endmodule
```

20. Write a verilog code to Synchronize async signal into clock domain.

```
module sync_2ff(input clk, async_in, output reg sync_out);
reg d1;
always @(posedge clk) begin
  d1 <= async_in;
  sync_out <= d1;
end
endmodule
```

21. Design a Reset Synchronizer (Active high async reset → sync release)

```
module reset_sync(input clk, arst, output reg srst);
reg r1;
always @(posedge clk or posedge arst) begin
 if(arst) begin
   r1 <= 1;
   srst <= 1;
 end
 else begin
   r1 <= 0;
   srst <= r1;
 end
end
endmodule
```

22. Write a code for Pulse Synchronizer between two clock domains in verilog.

```
module pulse_sync(
 input src_clk, dst_clk,
 input pulse_in,
 output reg pulse_out
);

reg toggle, sync1, sync2;

always @(posedge src_clk) begin
  if(pulse_in) toggle <= ~toggle;
end

always @(posedge dst_clk) begin
  sync1 <= toggle;
  sync2 <= sync1;
  pulse_out <= sync1 ^ sync2;
end

endmodule
```

23. Write a verilog code for a Signed 8×8 Multiplier (2's complement)

```
module signed_mult(input signed [7:0] a,b, output signed [15:0] y);
assign y = a * b;
endmodule
```

24. Design CDC safe asynchronous FIFO skeleton (also conceptually expected in interviews).

```
module async_fifo #(parameter DEPTH=8, WIDTH=8)(
 input wclk, rclk, rst,
 input wr_en, rd_en,
 input [WIDTH-1:0] din,
 output reg [WIDTH-1:0] dout
);

reg [WIDTH-1:0] mem [0:DEPTH-1];
reg [$clog2(DEPTH):0] wptr_bin, rptr_bin;

always @(posedge wclk) begin
 if(wr_en) begin
  mem[wptr_bin[$clog2(DEPTH)-1:0]] <= din;
  wptr_bin <= wptr_bin + 1;
 end
end

always @(posedge rclk) begin
 if(rd_en) begin
  dout <= mem[rptr_bin[$clog2(DEPTH)-1:0]];
  rptr_bin <= rptr_bin + 1;
 end
end

endmodule
```

25. Implement producer which asserts valid until ready is high using verilog

```
module axi_tx(input clk,rst,start,ready, output reg valid);
reg state;
parameter IDLE=0, SEND=1;

always @(posedge clk or posedge rst) begin
 if(rst) begin state<=IDLE; valid<=0; end
 else begin
  case(state)
   IDLE: if(start) begin state<=SEND; valid<=1; end
   SEND: if(ready) begin state<=IDLE; valid<=0; end
  endcase
 end
end
endmodule
```

26. Implement Priority-based round-robin arbiter for 8 masters using verilog

```
module arb8(input clk,rst, input [7:0] req, output reg [7:0] grant);
integer i;

always @(*) begin
 grant = 0;
 for(i=7;i>=0;i=i-1)
  if(req[i]) begin
   grant[i]=1;
   disable for;
  end
end
endmodule
```

27. Implement 32-bit population count (bit count) using verilog

```
module popcount(input [31:0] a, output reg [5:0] count);
integer i;

always @(*) begin
  count = 0;
  for(i=0;i<32;i=i+1)
    if(a[i]) count = count + 1;
end

endmodule
```

28. Generate PWM with programmable duty cycle using veriog

```
module pwm #(parameter N=8)(
 input clk,rst,
 input [N-1:0] duty,
 output reg pwm_out
);

reg [N-1:0] cnt;

always @(posedge clk or posedge rst) begin
 if(rst) begin cnt<=0; pwm_out<=0; end
 else begin
  cnt <= cnt + 1;
  pwm_out <= (cnt < duty);
 end
end
endmodule
```

29. Sequence detector for overlapping sequence "1101" (Moore) using verilog

```
module seq1101(input clk,rst,x, output reg y);
reg [2:0] state;
parameter S0=0,S1=1,S2=2,S3=3,S4=4;
always @(posedge clk or posedge rst) begin
 if(rst) begin state<=S0; y<=0; end
 else begin
  case(state)
    S0: begin y<=0; if(x) state<=S1; else state<=S0; end
    S1: begin y<=0; if(x) state<=S2; else state<=S0; end
    S2: begin y<=0; if(~x) state<=S3; else state<=S2; end
    S3: begin if(x) begin state<=S4; y<=1; end else begin state<=S0; y<=0; end end
    S4: begin y<=0; if(x) state<=S2; else state<=S0; end
  endcase
 end
end
endmodule
```

30. Parameterized Register File with Write-First Behavior using verilog

```
module regfile #(parameter WIDTH=32, DEPTH=8)(
 input clk,
 input we,
 input [$clog2(DEPTH)-1:0] waddr,raddr1,raddr2,
 input [WIDTH-1:0] wdata,
 output reg [WIDTH-1:0] rdata1,rdata2
);
reg [WIDTH-1:0] mem[0:DEPTH-1];
always @(posedge clk) begin
 if(we) mem[waddr] <= wdata;
 rdata1 <= mem[raddr1];
 rdata2 <= mem[raddr2];
end
endmodule
```

31. Write Verilog RTL for a configurable pipeline with N stages and synchronous reset.

```
module pipeline #(parameter STAGES=4, WIDTH=32)(
  input clk, rst,
  input  [WIDTH-1:0] din,
  output [WIDTH-1:0] dout
);

reg [WIDTH-1:0] pipe [0:STAGES-1];
integer i;
always @(posedge clk) begin
  if(rst) begin
    for(i=0;i<STAGES;i=i+1)
      pipe[i] <= 0;
  end else begin
    pipe[0] <= din;
    for(i=1;i<STAGES;i=i+1)
      pipe[i] <= pipe[i-1];
  end
end
assign dout = pipe[STAGES-1];
endmodule
```

32. Write RTL for clock gating using enable (conceptual—used to test understanding).

```
module clk_gate(input clk, en, output gclk);
reg latch_en;
always @(*) begin
  if(!clk)
    latch_en = en;
end
assign gclk = clk & latch_en;
endmodule
```

33. Create a simple memory-mapped register block with 4 registers.

```
module mmio(
 input clk,we,
 input [1:0] addr,
 input [31:0] wdata,
 output reg [31:0] rdata
);

reg [31:0] regfile [0:3];

always @(posedge clk)
 if(we) regfile[addr] <= wdata;

always @(*) begin
 rdata = regfile[addr];
end

endmodule
```

34. Count number of leading zeros in a 16-bit input.

```
module lzd16(input [15:0] a, output reg [4:0] count);
integer i;

always @(*) begin
  count = 0;
  for(i=15;i>=0;i=i-1) begin
    if(a[i]==0) count = count + 1;
    else disable for;
  end
end
endmodule
```

35. Implement 8-bit saturation addition using verilog.

```
module sat_add(
 input [7:0] a,b,
 output reg [7:0] y
);

wire [8:0] sum = a + b;

always @(*) begin
  if(sum[8]) y = 8'hFF;
  else       y = sum[7:0];
end
endmodule
```

36. Design FSM that takes 3 cycles to assert done using verilog

```
module multicycle(input clk,rst,start, output reg done);
reg [1:0] cnt;

always @(posedge clk or posedge rst) begin
 if(rst) begin cnt<=0; done<=0; end
 else if(start) begin cnt<=2'd3; done<=0; end
 else if(cnt!=0) begin cnt<=cnt-1; if(cnt==1) done<=1; end
 else done<=0;
end
endmodule
```

37. Trigger reset if not kicked within TIMEOUT using verilog

```
module watchdog #(parameter TIMEOUT=100)(
 input clk,rst,kick,
 output reg wdt_reset
);

reg [$clog2(TIMEOUT)-1:0] cnt;

always @(posedge clk or posedge rst) begin
 if(rst) begin cnt<=0; wdt_reset<=0; end
 else if(kick) begin cnt<=0; wdt_reset<=0; end
 else if(cnt==TIMEOUT-1) wdt_reset<=1;
 else cnt<=cnt+1;
end
endmodule
```

38. Track resource usage using busy bits using verilog

```
module scoreboard(
 input clk,
 input alloc, free,
 input [3:0] id,
 output busy
);
reg [15:0] table;

always @(posedge clk) begin
 if(alloc) table[id] <= 1;
 if(free)  table[id] <= 0;
end
assign busy = table[id];
endmodule
```

39. Implement 32-bit memory with 4 byte enables using verilog

```
module mem_be(
 input clk,we,
 input [3:0] be,
 input [31:0] wdata,
 output reg [31:0] rdata
);

always @(posedge clk) begin
 if(we) begin
  if(be[0]) rdata[7:0]   <= wdata[7:0];
  if(be[1]) rdata[15:8]  <= wdata[15:8];
  if(be[2]) rdata[23:16] <= wdata[23:16];
  if(be[3]) rdata[31:24] <= wdata[31:24];
 end
end
endmodule
```

40. Implement 1-deep skid buffer to handle back-pressure using verilog

```
module skid_buffer(
 input clk,rst,
 input valid_in,
 output ready_in,
 input [31:0] data_in,
 output reg valid_out,
 input ready_out,
 output reg [31:0] data_out
);

assign ready_in = ready_out | ~valid_out;
```

```
always @(posedge clk or posedge rst) begin
 if(rst) valid_out <= 0;
 else if(ready_in) begin
   valid_out <= valid_in;
   data_out  <= data_in;
 end
end
endmodule
```

41. Detect arithmetic overflow (signed) using verilog

```
module overflow_detect(
 input signed [7:0] a,b,
 input signed [7:0] sum,
 output overflow
);
assign overflow = (~(a[7]^b[7])) & (sum[7]^a[7]);
endmodule
```

42. Implement a CAM (content-addressable memory) using verilog

```
module cam #(parameter DEPTH=4, WIDTH=8)(
 input [WIDTH-1:0] key,
 input [WIDTH-1:0] mem [0:DEPTH-1],
 output reg hit
);
integer i;
always @(*) begin
 hit = 0;
 for(i=0;i<DEPTH;i=i+1)
   if(mem[i]==key) hit=1;
end
endmodule
```

43. Dynamic clock divider (runtime programmable) design using verilog

```
module prog_clk_div(
 input clk,rst,
 input [7:0] div,
 output reg clk_out
);
reg [7:0] cnt;

always @(posedge clk or posedge rst) begin
 if(rst) begin cnt<=0; clk_out<=0; end
 else if(cnt==div) begin cnt<=0; clk_out<=~clk_out; end
 else cnt<=cnt+1;
end
endmodule
```

44. Implement data-valid aligned pipeline using verilog

```
module valid_pipe(
 input clk,rst,
 input valid_in,
 input [31:0] din,
 output reg valid_out,
 output reg [31:0] dout
);

always @(posedge clk or posedge rst) begin
 if(rst) begin valid_out<=0; dout<=0; end
 else begin
  valid_out <= valid_in;
  dout <= din;
 end
end
endmodule
```

45. Multi-bit glitch-free enable synchronizer circuit using verilog

```
module en_sync(
 input clk,
 input [3:0] en_async,
 output reg [3:0] en_syncd
);
reg [3:0] d1;

always @(posedge clk) begin
 d1 <= en_async;
 en_syncd <= d1;
end
endmodule
```

46. Implement a minimal AXI-Lite slave supporting single-cycle read and write for 4 registers.

```
module axi_lite_slave (
 input clk, rst,
 input awvalid, wvalid, arvalid,
 input [1:0] awaddr, araddr,
 input [31:0] wdata,
 output reg awready, wready, arready,
 output reg rvalid, bvalid,
 output reg [31:0] rdata
);

reg [31:0] regfile [0:3];
```

```
always @(posedge clk or posedge rst) begin
 if(rst) begin
  awready<=0; wready<=0; arready<=0;
  rvalid<=0; bvalid<=0;
 end else begin
  awready <= awvalid;
  wready  <= wvalid;
  arready <= arvalid;

  if(awvalid && wvalid) begin
    regfile[awaddr] <= wdata;
    bvalid <= 1;
  end else bvalid <= 0;

  if(arvalid) begin
    rdata  <= regfile[araddr];
    rvalid <= 1;
  end else rvalid <= 0;
 end
end
endmodule
```

47. Design UART TX with start bit, 8 data bits, stop bit using verilog

```
module uart_tx #(parameter CLKS_PER_BIT=16)(
 input clk,rst,start,
 input [7:0] data,
 output reg tx, busy
);

reg [3:0] bit_cnt;
reg [7:0] shift;
reg [7:0] clk_cnt;
```

```
always @(posedge clk or posedge rst) begin
 if(rst) begin tx<=1; busy<=0; end
 else if(start && !busy) begin
  busy<=1; bit_cnt<=0; clk_cnt<=0;
  shift<=data; tx<=0;
 end else if(busy) begin
  if(clk_cnt==CLKS_PER_BIT-1) begin
   clk_cnt<=0;
   bit_cnt<=bit_cnt+1;
   case(bit_cnt)
    0: tx<=shift[0];
    1: tx<=shift[1];
    2: tx<=shift[2];
    3: tx<=shift[3];
    4: tx<=shift[4];
    5: tx<=shift[5];
    6: tx<=shift[6];
    7: tx<=shift[7];
    8: tx<=1;
    9: begin tx<=1; busy<=0; end
   endcase
  end else clk_cnt<=clk_cnt+1;
 end
end
endmodule
```

48. Implement UART RX with sampling and data reconstruction using verilog

```
module uart_rx #(parameter CLKS_PER_BIT=16)(
 input clk,rst,rx,
 output reg [7:0] data,
 output reg valid
);
reg [3:0] bit_cnt;
reg [7:0] clk_cnt;
```

```
always @(posedge clk or posedge rst) begin
 if(rst) begin valid<=0; bit_cnt<=0; end
 else begin
  if(rx==0 && bit_cnt==0) begin
    clk_cnt<=CLKS_PER_BIT/2;
    bit_cnt<=1;
  end else if(bit_cnt>0) begin
    if(clk_cnt==CLKS_PER_BIT-1) begin
      clk_cnt<=0;
      if(bit_cnt<=8) data[bit_cnt-1]<=rx;
      bit_cnt<=bit_cnt+1;
      if(bit_cnt==9) begin valid<=1; bit_cnt<=0; end
    end else clk_cnt<=clk_cnt+1;
  end
 end
end
endmodule
```

49. Design SPI master (CPOL=0, CPHA=0) using verilog

```
module spi_master(
 input clk,rst,start,
 input [7:0] data,
 output reg mosi,sclk,cs,busy
);

reg [2:0] bit_cnt;

always @(posedge clk or posedge rst) begin
 if(rst) begin cs<=1; sclk<=0; busy<=0; end
 else if(start && !busy) begin
  busy<=1; cs<=0; bit_cnt<=7;
 end else if(busy) begin
  sclk<=~sclk;
  if(sclk) begin
```

```
   mosi<=data[bit_cnt];
   if(bit_cnt==0) begin cs<=1; busy<=0; end
   else bit_cnt<=bit_cnt-1;
  end
 end
end
endmodule
```

50. Implement a FIFO with Almost-Full / Almost-Empty Flags using verilog

```
module fifo_af #(parameter DEPTH=16)(
 input clk,rst,wr,rd,
 output reg af,ae,
 output reg [$clog2(DEPTH):0] level
);

always @(posedge clk or posedge rst) begin
 if(rst) level<=0;
 else begin
  if(wr && !rd) level<=level+1;
  else if(rd && !wr) level<=level-1;
 end
end

always @(*) begin
 af = (level >= DEPTH-2);
 ae = (level <= 2);
end
endmodule
```

Excellence in World class VLSI Training & Placements

Do follow for updates & enquires

+91- 9182280927